



Guide 2: Every Project File Explained

Project: Personal Finance Classification Using Machine Learning

Purpose: Plain-English, comprehensive guide explaining **WHAT, HOW, WHY, and WHEN** every file and directory in this project is used.



Project Architecture Overview

— streamlit_app.py	# 1. Main interactive web application
— src/	
— predictor.py	# 2. Inference engine & financial advisor
— train_models.py	# 3. Model training & benchmarking pipeline
— eda.py	# 4. Exploratory Data Analysis generator
— data/	
— load_online_dataset.py	# 5. Online Kaggle dataset ingestion
— personal_finance_data.csv	# 6. Standardized 20,000-sample dataset
— dataset_source.json	# 7. Dataset origin & provenance metadata
— eda_summary.json	# 8. Precomputed class statistics
— models/	
— decision_tree_model.joblib	# 9. Serialized Decision Tree model
— random_forest_model.joblib	# 10. Serialized Random Forest model
— gradient_boosting_model.joblib	# 11. Serialized Gradient Boosting model
— logistic_regression_model.joblib	# 12. Serialized Logistic Regression model
— scaler.joblib	# 13. Serialized StandardScaler
— label_encoder.joblib	# 14. Serialized LabelEncoder
— metrics.json	# 15. Benchmark performance scores
— reports/figures/	# 16. High-res evaluation & EDA charts
— notebooks/	
— Personal_Finance_Classification.ipynb	# 17. Complete Jupyter Case Study Notebook
— generate_notebook.py	# 18. Programmatic notebook generator
— docs/	# 19. Comprehensive educational documentation
— 1_ML_TOPICS_EXPLAINED.md	# Guide to all ML concepts
— 2_PROJECT_FILES_EXPLAINED.md	# Guide to all files (this file)
— 3_ML_MODELS_EXPLAINED.md	# Guide to the 4 best models
— requirements.txt	# 20. Python dependencies for deployment
— runtime.txt & .python-version	# 21. Python 3.11 runtime pinning
— .streamlit/config.toml	# 22. Streamlit UI theme configuration
— PROJECT_REPORT.md	# 23. Academic case study report
— README.md	# 24. Project summary & GitHub homepage

1. `streamlit_app.py`

What is it?

The **heart and soul of the user experience**. This is a modern, responsive web application that runs both locally and on Streamlit Cloud, allowing users, professors, or evaluators to interact directly with the machine learning models.

HOW does it work?

1. It imports `src/predictor.py` to load the models into memory via `@st.cache_resource`.
2. It organizes the experience into **4 distinct application views** via a sidebar radio menu: - ⚡ **Live Classifier Studio**: Interactive form with sliders, number inputs, and 3 one-click preset buttons (*Disciplined Saver, Moderate Balanced, Distressed High-Spender*). - 📊 **Model Comparative Study**: Comprehensive performance table, comparative bar chart, 2x2 confusion matrix grid, and feature importance rankings. - 🔍 **EDA & Spending Patterns**: Deep visual exploration of 20,000 consumer records across 6 high-resolution charts. - 📄 **Academic Final Analysis**: Formal executive summary, behavioral findings, model justifications, real-world limitations, and commercial FinTech applications.
3. When the user clicks **"Run Financial Classification"**, it calls `service.predict()`, renders dynamic archetype badges with glowing color codes (green, blue, red), plots an **interactive 50/30/20 budget chart via Altair**, and displays tailored bullet-point recommendations.

WHY did we build it?

A machine learning model locked in a command-line script is invisible. Streamlit provides a professional, zero-latency, full-stack interface that allows non-technical stakeholders to test the model in real time.

WHEN is it used?

- Used during live demonstrations, user testing, grading evaluation, and production deployment on Streamlit Cloud.
- Run locally with: `streamlit run streamlit_app.py`

2. `src/predictor.py`

What is it?

The **production inference engine and financial advisory brain**. It contains the class `FinancialClassifierService`, which bridges the gap between raw user input and model output.

HOW does it work?

1. **Model Loading:** Reads `models/metrics.json` to identify available models, then loads the `.joblib` files into a Python dictionary.
2. **Feature Engineering on the Fly:** Takes the raw user dictionary (e.g., `{'monthly_income': 50000, 'monthly_expenses': 25000, ...}`) and instantly computes the 6 financial ratios: -
 $\text{savings_rate} = (\text{savings} / \text{monthly_income}) * 100$ - $\text{expense_to_income} = (\text{expenses} / \text{monthly_income}) * 100$ - $\text{debt_to_income} = (\text{loans} / \text{monthly_income}) * 100$ -
 $\text{discretionary_ratio} = (\text{entertainment} + \text{shopping}) / \text{expenses} * 100$ - $\text{essential_ratio} = (\text{housing} + \text{food} + \text{transport} + \text{health}) / \text{expenses} * 100$ - $\text{investment_rate} = (\text{investments} / \text{monthly_income}) * 100$
3. **Scaling:** Uses `scaler.transform()` if the selected model requires scaled inputs (e.g., Logistic Regression).
4. **Prediction & Confidence:** Calls `model.predict()` and `model.predict_proba()`.
5. **Financial Health Score (0–100):** Calculates an objective composite credit-health index: - Base = 50 points - +30 points based on savings rate - –25 points if expense ratio exceeds 70% - –20 points if debt-to-income exceeds 15% - –15 points if credit card utilization exceeds 40%
6. **Smart Recommendations:** Evaluates financial risk triggers and outputs categorized advice tags (`[EXPENSE CONTROL]`, `[DEBT ACCELERATION]`, `[WEALTH ACCELERATION]`).
7. **Cloud Self-Healing Fallback (`_train_in_environment`):** If Streamlit Cloud runs on a newer Python or Scikit-Learn version that cannot unpickle an artifact created on a different computer, `predictor.py` automatically detects the issue, retrains the 4 models in under 5 seconds, saves fresh local artifacts, and proceeds without crashing!

WHY did we build it?

Separation of Concerns. The UI (`streamlit_app.py`) should only handle presentation, while the inference and business logic live in `predictor.py`. This allows the exact same predictor to power a mobile app, web app, or command-line script.

WHEN is it used?

- Called every single time a classification is requested in the Streamlit app or API.

3. `src/train_models.py`

What is it?

The **offline machine learning laboratory**. This script trains, cross-validates, evaluates, and serializes the 4 best classification algorithms on the 20,000-sample dataset.

HOW does it work?

1. **Data Ingestion:** Loads `data/personal_finance_data.csv`.
2. **Train/Test Splitting:** Executes an 80/20 stratified split (16,000 training records, 4,000 test records) with `random_state=42`.
3. **Scaling:** Fits `StandardScaler` on the training set and transforms both sets.
4. **Multi-Model Training Loop:** - Decision Tree (`max_depth=6`) - Random Forest (`n_estimators=100`, `max_depth=10`) - Gradient Boosting (`n_estimators=80`, `learning_rate=0.1`) - Logistic Regression (`C=1.0`, `max_iter=1000`)
5. **5-Fold Cross Validation:** Runs `StratifiedKFold(n_splits=5)` to test stability across 5 distinct data slices.
6. **Metric Calculation:** Evaluates Accuracy, Macro Precision, Macro Recall, Macro F1, Weighted F1, and per-class metrics.
7. **Visualization Generation:** Creates and saves: - `reports/figures/model_comparison_bar.png` - `reports/figures/confusion_matrices_grid.png` (2x2 grid) - `reports/figures/feature_importances.png`
8. **Artifact Export:** Saves the 4 models, scaler, label encoder, and `metrics.json` to the `models/` directory.

WHY did we build it?

Training takes computational time and memory. In real-world machine learning, training is done offline on powerful servers, while lightweight saved models are deployed to web servers.

WHEN is it used?

- Run whenever new data is added, hyperparameters are tweaked, or models need to be retrained: `python3 src/train_models.py`.

4. `src/eda.py`

What is it?

The **automated data intelligence pipeline**. It conducts rigorous exploratory data analysis on the 20,000 consumer records and outputs high-resolution visualization figures and statistical summaries.

HOW does it work?

1. Loads the dataset and computes all derived financial ratios.

- Generates 6 publication-grade figures saved to `reports/figures/` : -
`eda_class_distribution.png` : Bar and pie charts of archetype counts. -
`eda_financial_ratios.png` : Boxplots showing distributions of savings, expenses, DTI, and discretionary ratios. - `eda_spending_breakdown.png` : Stacked bar chart showing what percentage of expenditures go to housing, food, transport, healthcare, entertainment, and shopping. - `eda_income_vs_expense.png` : Scatter plot proving income neutrality. -
`eda_correlation_matrix.png` : Heatmap of Pearson correlation coefficients. -
`eda_discretionary_vs_savings.png` : Trade-off plot between luxury spending and savings rate.
- Computes aggregate statistical averages per archetype and exports them to `data/eda_summary.json` .

WHY did we build it?

Writing ad-hoc plotting code inside a notebook makes it difficult to reuse charts in web dashboards. Having a dedicated script ensures that anytime the dataset updates, all 6 figures and summary statistics can be regenerated in 3 seconds with a single command.

WHEN is it used?

- Run to regenerate analytical figures: `python3 src/eda.py` .

5. `data/load_online_dataset.py`

What is it?

The **data ingestion and standardization pipeline**. It downloads the authentic, real-world *Indian Personal Finance and Spending Habits Dataset* from Kaggle/GitHub, cleans it, maps it into standard features, and exports it to CSV.

HOW does it work?

- Fetches raw data containing 20,000 real individual financial records.
- Standardizes column names (e.g., `Monthly_Income` → `monthly_income` , `Housing_Utilities` → `housing_utilities`).
- Classifies profiles into the target archetypes (**Saver**, **Balanced**, **High-Spender**) based on their verified financial indicators.
- Audits and drops any missing or anomalous records.
- Saves the clean, standardized dataset to `data/personal_finance_data.csv` .

WHY did we build it?

Synthetic, artificially made-up data lacks real-world variance and realistic behavioral noise. Ingesting this real 20,000-sample dataset guarantees that our findings reflect genuine consumer financial habits.

WHEN is it used?

- Run once during project setup to ingest the dataset: `python3 data/load_online_dataset.py`.

6. `data/personal_finance_data.csv`

What is it?

The **canonical master dataset** of the project. - **Dimensions:** 20,000 rows × 13 columns. - **Columns:** - `monthly_income` : Gross monthly earnings (₹). - `monthly_expenses` : Total monthly spending (₹). - `savings` : Monthly savings allocation (₹). - `loan_payments` : Total monthly loan, mortgage, or EMI commitments (₹). - `investment_amount` : Monthly stock, mutual fund, or deposit allocations (₹). - `housing_utilities` : Rent, maintenance, electricity, water (₹). - `food_dining` : Groceries, dining out, food delivery (₹). - `transportation` : Fuel, public transit, cab fares (₹). - `healthcare` : Medical insurance, prescriptions, doctor visits (₹). - `entertainment` : Movies, events, streaming subscriptions, leisure (₹). - `shopping_discretionary` : Electronics, apparel, personal luxury (₹). - `credit_card_utilization` : Percentage of credit card limit utilized (%). - `financial_category` : Target label (`Saver` , `Balanced` , `High-Spender`).

WHY is it important?

Every single model, chart, cross-validation split, and feature importance plot in this project is directly derived from this file.

7. `data/dataset_source.json`

What is it?

The **provenance and attribution record**. It documents: - Dataset name: *Indian Personal Finance and Spending Habits* - Source URLs: [Kaggle Dataset](#) & [GitHub Repository](#) - Author attribution, license, and ingestion timestamp.

WHY did we include it?

Academic rigor. It proves that the project uses an authentic open-source dataset with transparent citations.

8. `data/eda_summary.json`

What is it?

A lightweight JSON file containing **precomputed statistical summaries** for each archetype: - Total records per class - Mean savings rate per archetype (Saver: 36.66%, Balanced: 26.28%, High-Spender: 13.48%) - Mean debt-to-income ratio (Saver: 0.44%, Balanced: 3.60%, High-Spender: 11.53%) - Mean expense-to-income ratio - Average monthly earnings

WHY did we build it?

Instead of forcing `streamlit_app.py` to reload a 20,000-row CSV file and compute heavy group-by aggregations on every user visit, the app reads this 2KB JSON file instantly, ensuring sub-second page loads.

9–15. The `models/` Directory

Contains all persistent machine learning artifacts: - `decision_tree_model.joblib` : The trained Decision Tree Classifier (Best Model, 99.98% accuracy). - `random_forest_model.joblib` : The trained Random Forest Classifier (100 trees, 99.98% accuracy). - `gradient_boosting_model.joblib` : The trained Gradient Boosting Classifier (80 trees, 99.98% accuracy). - `logistic_regression_model.joblib` : The trained Logistic Regression Classifier (Softmax, 99.62% accuracy). - `scaler.joblib` : The fitted `StandardScaler` storing column means (μ) and standard deviations (σ). - `label_encoder.joblib` : The fitted `LabelEncoder` mapping text labels to numbers. - `metrics.json` : Structured scorecard containing test accuracies, 5-fold CV means, standard deviations, macro/weighted F1 scores, and confusion matrix arrays for all 4 models.

16. `reports/figures/`

Contains all 9 exported high-resolution PNG plots: 1. `model_comparison_bar.png` : Multi-metric bar chart comparing all 4 models. 2. `confusion_matrices_grid.png` : 2x2 grid of confusion matrices. 3. `feature_importances.png` : Gini importance bar chart (Random Forest vs. Gradient Boosting). 4. `eda_class_distribution.png` : Class balance pie chart and frequency bars. 5.

`eda_financial_ratios.png` : 4-panel boxplots of financial ratios. 6. `eda_spending_breakdown.png` : Category spending distribution by class. 7. `eda_income_vs_expense.png` : Income vs. expense scatter visualization. 8. `eda_correlation_matrix.png` : Full correlation heatmap. 9. `eda_discretionary_vs_savings.png` : Discretionary spend vs savings rate trade-off.

17–18. The `notebooks/` Directory

- `Personal_Finance_Classification.ipynb` : A complete, beautifully structured Jupyter Notebook covering all 8 steps of the case study (Problem Statement, Data Ingestion, EDA, Feature Engineering, Model Training, Cross-Validation, Comparative Evaluation, and Deployment Simulation).
 - `generate_notebook.py` : An automated Python utility that writes the entire `.ipynb` file in JSON format. This ensures the notebook can be updated programmatically whenever code changes, without needing to manually click through Jupyter cells.
-

19. The `docs/` Directory

Contains 3 exhaustive, simple-English educational guides: - `1_ML_TOPICS_EXPLAINED.md` : Explains every ML concept used (How, Why, When). - `2_PROJECT_FILES_EXPLAINED.md` : Explains every file in the project (this guide). - `3_ML_MODELS_EXPLAINED.md` : Explains each of the 4 best classification models in depth (How, Why, When).

20–22. Deployment Configuration Files

- `requirements.txt` : Specifies all Python libraries required by Streamlit Cloud: `streamlit`, `scikit-learn`, `pandas`, `numpy`, `matplotlib`, `seaborn`, `joblib`, and `altair`.
 - `runtime.txt` & `.python-version` : Pins the deployment runtime to **Python 3.11** to prevent compatibility issues with Python 3.14.
 - `.streamlit/config.toml` : Configures Streamlit's dark-mode UI palette, custom fonts, and server ports.
-

23–24. Documentation Files

- **PROJECT_REPORT.md** : Formal academic case study documentation formatted for semester review and viva evaluation.
 - **README.md** : Clean, elegant project presentation with setup instructions, feature lists, and quick-start commands.
-

Summary: You now have a complete, clear mental map of every single file in the repository and why it exists.